



UITs
UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES

PROJECT REPORT

ON

CAR RACING — OBSTACLE DODGE DRIVE

(Implemented Using OpenGL, C++)

Department : CSE

Course Code : CSE 358

Course Title : Computer Graphics and Multimedia Lab

Program : BSc in Computer Science & Engineering

Prepared by:

1. Name: Md. Tanvir Rahman

ID: 0432320005101116

2. Name: Faisal Ahmed

ID: 0432320005101129

3. Name: Md. Sayed

ID: 0432320005101148

Submitted To:

Audity Ghosh ,

Department of Computer Science and

Engineering

Submission Date: 29/06/2026

Table of Contents

1. Introduction and Motivation.....	3
1.1 Motivation.....	3
1.2 Objectives.....	3
2. Dataset / Game Feature Details.....	4
2.1 Window and Rendering Parameters.....	4
2.2 Player Parameters.....	4
2.3 Enemy Parameters.....	5
2.4 Difficulty Scaling.....	5
2.5 Game Controls.....	5
2.6 Game States.....	6
2.7 LearnOpenGL Chapters Applied.....	6
3. Methodology.....	6
3.1 System Architecture Overview.....	6
3.2 Block Diagram.....	7
3.3 Rendering Pipeline.....	7
3.4 Collision Detection.....	8
4. Pseudocode.....	8
4.1 Initialization.....	8
4.2 Main Game Loop.....	8
4.3 Update Player.....	9
4.4 Update Road.....	9
4.5 Update Enemies.....	9
4.6 Update Score.....	10
4.7 Render.....	10
5. Game Terminal Screenshots with Output.....	11
5.1 Console / Terminal Output on Startup.....	11
5.2 Game Screen States.....	11
6. Discussion and Future Work.....	11
6.1 Discussion.....	11
6.2 Limitations.....	12
6.3 Future Work.....	12
7. Conclusion.....	13
References.....	14

1. Introduction and Motivation

Computer Graphics is one of the most dynamic and visually rewarding areas of Computer Science. The ability to render real-time interactive scenes using modern GPU pipelines opens a wide range of creative and practical applications — from scientific visualization to interactive entertainment. The project "Obstacle Dodge Drive" was conceived as both a practical demonstration of core Modern OpenGL concepts and an engaging interactive game experience.

The game is a 2D top-down car dodging game in which the player navigates a speeding vehicle through oncoming traffic on a continuously scrolling road. The objective is to survive as long as possible while avoiding collision with enemy vehicles that spawn at increasing speeds and densities.

1.1 Motivation

The primary motivation for this project was to apply, in a cohesive and functional software product, the rendering concepts taught through the LearnOpenGL tutorial series (<https://learnopengl.com>). Rather than implementing isolated examples, the team chose to build a complete game that exercises the full pipeline — from window creation and OpenGL context initialization, through shader compilation and geometry setup, to texturing, transformations, coordinate systems, and alpha blending.

A secondary motivation was to demonstrate that high-quality interactive graphics can be achieved without reliance on heavyweight game engines or third-party asset libraries. Every visual element in Obstacle Dodge Drive — road, trees, cars, HUD, score digits — is drawn programmatically using OpenGL primitives. No image files are loaded from disk; even car textures are generated in CPU memory and uploaded to the GPU at runtime.

From an academic standpoint, this project bridges the gap between theoretical lecture material and hands-on implementation, reinforcing concepts such as the Model-View-Projection matrix pipeline, axis-aligned bounding box (AABB) collision detection, frame-rate-independent game loops, and state machine design.

1.2 Objectives

- Implement a fully functional 2D game using Modern OpenGL 3.3 Core Profile.
- Apply LearnOpenGL chapters on Window creation, Hello Triangle, Shaders, Textures, Transformations, Coordinate Systems, and Blending.
- Demonstrate procedural texture generation without external image files.
- Implement a 7-segment style score display using only OpenGL rectangle primitives.
- Design a clean game state machine covering Menu, Playing, Paused, and Game Over states.
- Achieve frame-rate-independent gameplay through delta-time normalization.

2. Dataset / Game Feature Details

Unlike data-driven machine learning projects, a real-time game project does not rely on an external dataset. Instead, the "dataset" for Obstacle Dodge Drive is composed of procedurally generated graphical assets, hard-coded game parameters, and runtime-randomized values that govern enemy behavior. This section documents all key feature parameters and their roles in the game.

2.1 Window and Rendering Parameters

Parameter	Value	Description
SCR_WIDTH	500 px	Game window width in pixels
SCR_HEIGHT	700 px	Game window height in pixels
OpenGL Version	3.3 Core	Modern OpenGL context profile used
Projection	glm::ortho	2D orthographic projection (pixel coords)
Texture Size	16×24 pixels	Procedural car texture resolution

2.2 Player Parameters

Parameter	Value	Description
Initial X Position	250 (center)	Horizontal starting position
Y Position	80 px	Fixed vertical position near bottom
Car Width (PW)	38 px	Half-width for collision and drawing

Car Height (PH)	62 px	Half-height for collision and drawing
Move Speed	11 units/frame	Normalized to 60 FPS via delta-time
Move Bounds	138 – 362 px	Horizontal movement limits (road edges)

2.3 Enemy Parameters

Parameter	Value	Description
Lane Positions	162, 250, 338 px	Three lanes on the road (X coordinates)
Base Speed	3.5 units/frame	Initial enemy downward speed
Speed Variance	+0 to +2.1	Random speed offset per enemy
Spawn Interval	120 → 45 frames	Decreases as difficulty scales
Despawn Y	< -100 px	Enemy removed when off-screen below

2.4 Difficulty Scaling

The game difficulty increases automatically as the player's score rises. Every 30 normalized frames, the score increments by 1, and the following scaling formulas apply:

```
baseSpeed = 3.5 + score * 0.065    // enemy base speed
laneSpeed = 6.5 + score * 0.11     // road scroll speed (capped at 28)
spawnInterval decreases by 0.4 each spawn until it reaches 45
```

2.5 Game Controls

Key	Action
Left Arrow (←)	Move player car to the left
Right Arrow (→)	Move player car to the right
SPACE	Start game from Menu or restart after Game Over
P	Pause the game / Resume from Pause
ESC	Exit the application

2.6 Game States

The game operates as a four-state finite state machine:

- MENU — The start screen displaying decorative cars and a high score.
- PLAYING — Active gameplay with movement, enemy spawning, collision detection, and score increment.
- PAUSED — Gameplay frozen; a pause overlay with visual indicators is rendered.
- GAME_OVER — Final score displayed with a tilted crashed car; awaits restart input.

2.7 LearnOpenGL Chapters Applied

#	Chapter	Applied In
1	Getting Started / Creating a Window	GLFW window setup, GLAD initialization
2	Getting Started / Hello Triangle	VAO, VBO — unit quad geometry
3	Getting Started / Shaders	Vertex & Fragment shader compile, link, uniforms
4	Getting Started / Textures	Procedural car texture via glTexImage2D
5	Getting Started / Transformations	glm::translate, glm::scale, glm::rotate — model matrix
6	Getting Started / Coordinate Systems	glm::ortho — 2D orthographic projection
7	Advanced OpenGL / Blending	Semi-transparent overlay with alpha blending

3. Methodology

The development of Obstacle Dodge Drive followed a structured pipeline typical of real-time rendering applications. The methodology is divided into initialization, the per-frame game loop, and shutdown. Each stage leverages specific OpenGL and GLFW APIs as learned through the LearnOpenGL tutorial series.

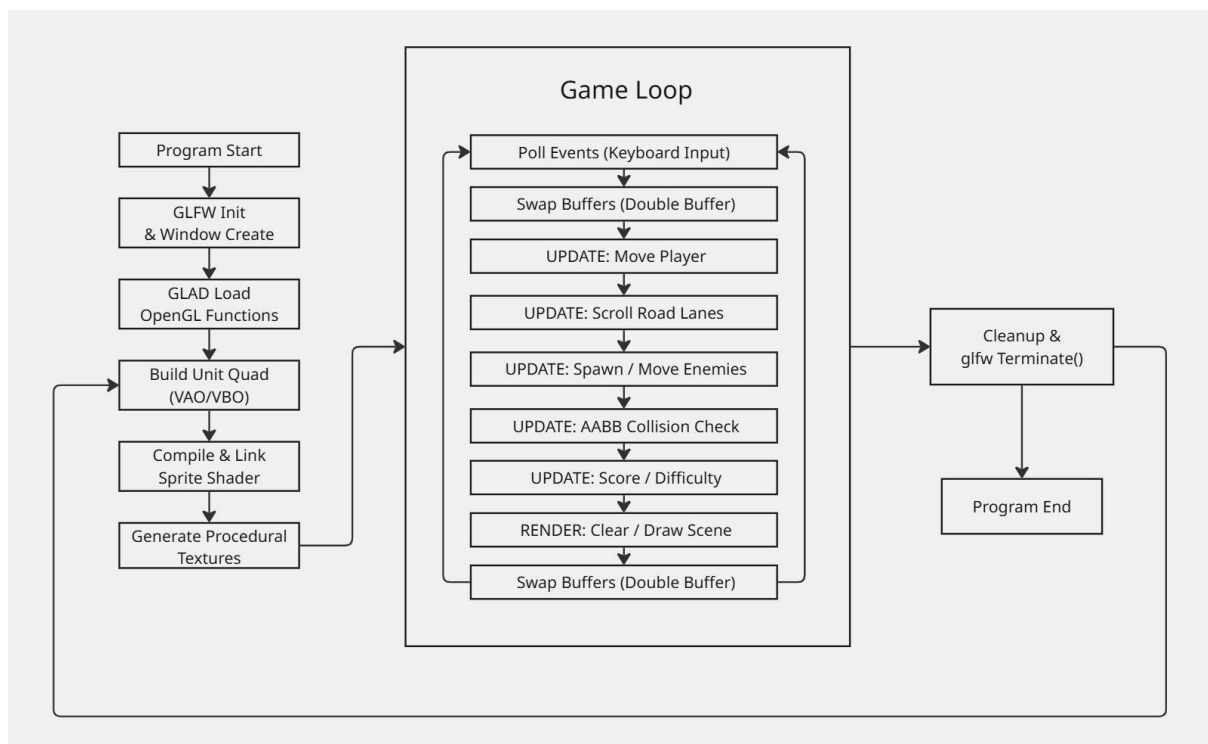
3.1 System Architecture Overview

The system is implemented as a single-file C++ application (main.cpp) complemented by the GLAD OpenGL loader (glad.c). The architecture is structured around the following key subsystems:

- Window and Context Management — GLFW handles platform-specific window creation and OpenGL context initialization.
- Shader System — A single sprite shader supports both flat-color and textured rendering via a boolean uniform switch (uUseTexture).
- Geometry System — A single unit quad $(0,0) \rightarrow (1,1)$ is uploaded once to the GPU and reused for every draw call; position and size are encoded in the model matrix.
- Texture System — Car textures are generated procedurally in CPU memory and uploaded to GPU via `glTexImage2D`.
- Game Logic — Update step handles player input, lane scrolling, enemy spawning, collision detection, and score scaling.
- Render System — Draw step renders all game objects in correct layer order: road $\rightarrow$ enemies $\rightarrow$ player $\rightarrow$ HUD $\rightarrow$ overlay.

3.2 Block Diagram

The following block diagram illustrates the high-level flow of the game loop:



Miro Diagram Link:

https://miro.com/app/board/uXjVHAgHnHLA=?share_link_id=238279285558

3.3 Rendering Pipeline

Every visual element is drawn as a scaled, translated, optionally rotated unit quad. The `drawSprite()` function constructs the model matrix using GLM, uploads it as a uniform to the vertex shader, and issues a single `glDrawArrays` call. The vertex shader applies the

Model-Projection transform; the fragment shader chooses between flat color and texture sampling based on the `uUseTexture` uniform.

3.4 Collision Detection

Axis-Aligned Bounding Box (AABB) collision detection compares the absolute distance between the player's center position and each enemy's center position against the sum of their half-extents in both the X and Y axes. A collision is confirmed when both conditions are simultaneously true:

```
bool checkCollision(const Enemy& e) {
    return fabsf(playerX - e.x) < PW + 26
        && fabsf(playerY - e.y) < PH + 45;
}
```

4. Pseudocode

The following pseudocode describes the high-level logic of the game:

4.1 Initialization

```
PROCEDURE Initialize:
    glfwInit()
    Set OpenGL version hints to 3.3 Core Profile
    Create window (500 x 700)
    gladLoadGLLoader()           // load OpenGL function pointers
    Enable global GL_BLEND
    Compile and link sprite shader
    Build unit quad VAO/VBO
    Generate procedural player texture
    Generate procedural enemy texture
    Record lastTime = glfwGetTime()
END PROCEDURE
```

4.2 Main Game Loop

```
WHILE window is not closed DO:
    now = glfwGetTime()
    deltaTime = now - lastTime
    lastTime = now
    glfwPollEvents()           // handle keyboard callbacks

    IF gameState == PLAYING THEN
        dt60 = deltaTime * 60.0    // normalize to 60 FPS
        CALL UpdatePlayer(dt60)
        CALL UpdateRoad(dt60)
        CALL UpdateEnemies(dt60)
        CALL UpdateScore(dt60)
    END IF
```



```

    glClearColor(COLOR_BUFFER_BIT)
    CALL Render()
    glfwSwapBuffers(window)
END WHILE

```

4.3 Update Player

```

PROCEDURE UpdatePlayer(dt60):
    IF keyLeft AND playerX > 138 THEN playerX -= 11 * dt60
    IF keyRight AND playerX < 362 THEN playerX += 11 * dt60
END PROCEDURE

```

4.4 Update Road

```

PROCEDURE UpdateRoad(dt60):
    FOR each lane marker i in [0..5] DO
        laneY[i] -= laneSpeed * dt60
        IF laneY[i] < -40 THEN laneY[i] += 740 // wrap to top
    END FOR
END PROCEDURE

```

4.5 Update Enemies

```

PROCEDURE UpdateEnemies(dt60):
    spawnTimer += dt60
    IF spawnTimer >= spawnInterval THEN
        SpawnEnemy()
        spawnTimer = 0
        IF spawnInterval > 45 THEN spawnInterval -= 0.4
    END IF

    FOR each enemy e (reversed order for safe removal) DO
        e.y -= e.speed * dt60
        IF CheckCollision(e) THEN
            gameState = GAME_OVER
            IF score > highScore THEN highScore = score
        END IF
        IF e.y < -100 THEN REMOVE e from enemies list
    END FOR
END PROCEDURE

```

```

PROCEDURE SpawnEnemy():
    e.x = laneXPos[ random(0,2) ]
    e.y = SCR_HEIGHT + 80
    e.speed = baseSpeed + random(0,7) * 0.3
    e.color = random warm color
    APPEND e to enemies list
END PROCEDURE

```

```

FUNCTION CheckCollision(e) → BOOLEAN:
    RETURN |playerX - e.x| < PW + 26

```

```
AND |playerY - e.y| < PH + 45
```

4.6 Update Score

```
PROCEDURE UpdateScore(dt60):
    scoreTimer += dt60
    IF scoreTimer >= 30 THEN
        score += 1
        scoreTimer = 0
        baseSpeed = 3.5 + score * 0.065
        laneSpeed = min(6.5 + score * 0.11, 28.0)
    END IF
END PROCEDURE
```

4.7 Render

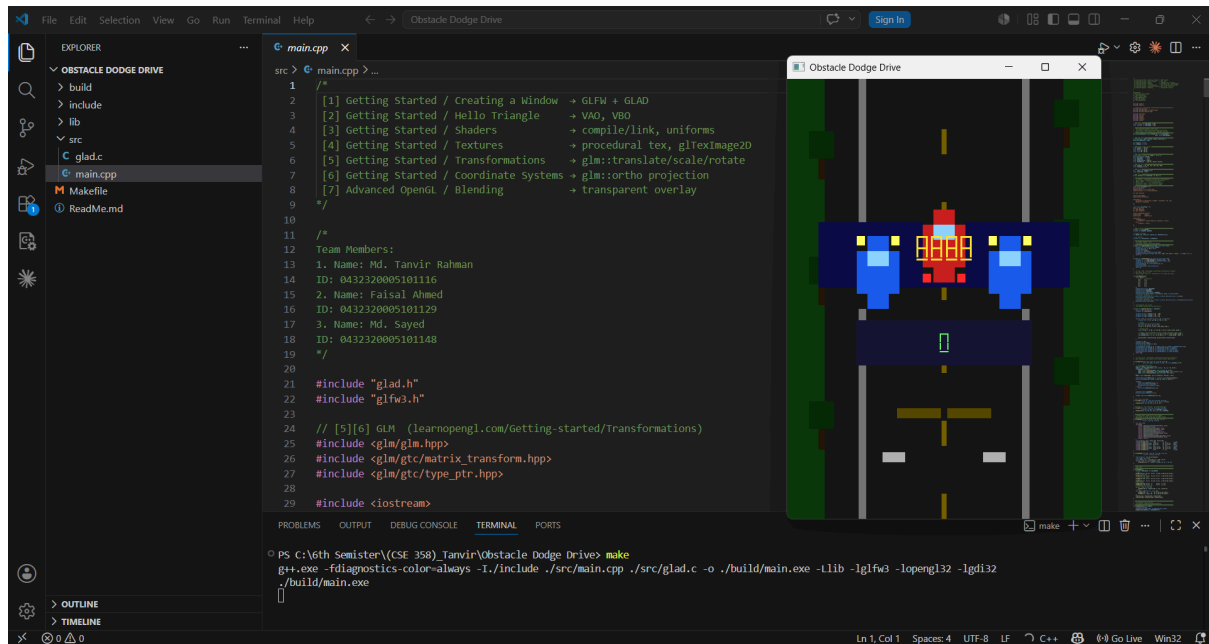
```
PROCEDURE Render():
    CALL DrawRoad() // sky, grass, asphalt, lane marks, trees

    IF gameState == MENU THEN
        CALL DrawOverlay(alpha=0.55)
        CALL DrawMenuDecorations()

    ELSE IF gameState IN {PLAYING, PAUSED, GAME_OVER} THEN
        FOR each enemy e DO DrawCar(e.x, e.y, enemyTex)
        DrawCar(playerX, playerY, playerTex)
        CALL DrawHUD() // score bar, speed bar

        IF gameState == PAUSED THEN DrawOverlay(0.50) + DrawPauseSymbol()
        IF gameState == GAME_OVER THEN DrawOverlay(0.65) + DrawGameOverPanel()
    END IF
END PROCEDURE
```

5. Game Terminal Screenshots with Output



6. Discussion and Future Work

6.1 Discussion

The Obstacle Dodge Drive project successfully demonstrates a complete real-time rendering pipeline using Modern OpenGL 3.3 Core Profile. All seven targeted LearnOpenGL chapters were applied in the implementation, and the game achieves its primary objectives: smooth player movement, procedurally generated visuals, difficulty scaling, and robust game state management.

One of the most noteworthy technical achievements is the elimination of all external asset dependencies. By generating car textures procedurally (16×24 pixel arrays computed in CPU memory and uploaded via `glTexImage2D`), the project avoids `stb_image` or similar loaders entirely. The 7-segment score display — built from individual `drawRect` calls mapping to segment enable bitmasks — similarly avoids font libraries such as FreeType.

Frame-rate independence is achieved by multiplying all movement and spawn quantities by $dt60 = \text{deltaTime} * 60.0$, ensuring identical gameplay behavior at 30 FPS, 60 FPS, or 144 FPS. This is a standard technique in commercial game development and its implementation here demonstrates a solid understanding of real-time game loop design.

The single-shader approach (toggling between flat-color and textured modes via a boolean uniform) simplifies the codebase but does impose a minor GPU overhead from the conditional branch per fragment. For a 2D game of this scale this is entirely negligible; in a larger 3D scene it would warrant separate specialized shaders.

AABB collision detection is efficient and appropriate for this game's rectangular sprites aligned to the screen axes. The padded half-extents (PW+26, PH+45) provide a slightly forgiving collision margin that improves perceived fairness.

6.2 Limitations

- No sound or music — the game is entirely silent. Audio integration (e.g., via OpenAL) would significantly enhance immersion.
- No persistent high score — the session high score resets on application exit. A file-based or registry-based save system is absent.
- The 7-segment display cannot render letters, limiting textual UI to numeric values only.
- Enemy cars use fixed lane positions; there is no diagonal movement or lane-changing behavior.
- The game has no level structure — difficulty increases are continuous but there are no discrete stages or boss encounters.

6.3 Future Work

Several enhancements are planned or recommended for future iterations:

1. Audio System — Integrating OpenAL or a simple audio library to add engine sounds, collision sound effects, and background music.
2. Persistent High Score — Save the high score to a local file using standard C++ file.
3. Font Rendering — Implementing FreeType-based text rendering to display alphabetic labels ("SCORE", "BEST", "PAUSED", "GAME OVER") directly on screen.
4. Additional Car Designs — Expand the procedural texture generator to produce variety in car shapes (trucks, motorcycles, sports cars) using more complex pixel patterns.
5. Difficulty Modes — Add selectable difficulty modes (Easy / Normal / Hard) that adjust initial baseSpeed, spawnInterval, and scaling coefficients.
6. Mobile / Web Port — Explore porting the game to OpenGL ES 2.0 (Android/iOS) or WebGL (browser) to broaden accessibility.

7. Conclusion

Obstacle Dodge Drive is a complete, functional, and visually polished 2D game developed entirely with Modern OpenGL 3.3 Core Profile and C++17, without any game engine or external asset dependencies. The project was built as the capstone deliverable for the Computer Graphics and Multimedia Laboratory course (CSE 358) and successfully applied all seven targeted chapters from the LearnOpenGL tutorial series.

The game demonstrates mastery of the fundamental OpenGL rendering pipeline, including context initialization with GLFW and GLAD, VAO/VBO geometry upload, GLSL shader compilation and uniform communication, procedural texture generation with `glTexImage2D`, the Model-View-Projection matrix stack using GLM, 2D orthographic projection, and alpha blending for transparent overlays.

Beyond pure graphics, the project also implements core game development patterns: a delta-time normalized game loop for frame-rate independence, a four-state finite state machine, AABB collision detection, dynamic difficulty scaling, and session high score tracking.

The project affirms that compelling interactive applications can be constructed from first principles using only low-level graphics APIs, deepening the team's understanding of how modern GPUs process and render geometry. The experience gained through this project provides a solid foundation for future work in 3D rendering, game engine architecture, and graphics programming.

References

- de Vries, J. (n.d.). LearnOpenGL. Retrieved June 23, 2026, from <https://learnopengl.com/>
- Kessenich, J., Sellers, G., & Shreiner, D. (2016). OpenGL programming guide: The official guide to learning OpenGL, version 4.5 (9th ed.). Addison-Wesley.
- GLFW Development Team. (2023). GLFW: An OpenGL library (Version 3.x) [Software]. Retrieved from <https://www.glfw.org/>
- G-Truc Creation. (2023). OpenGL Mathematics (GLM) [Software library]. Retrieved from <https://glm.g-truc.net/>
- Khronos Group. (2023). OpenGL 3.3 core profile specification. Retrieved from <https://www.khronos.org/opengl/wiki/>
- GLAD Contributors. (2023). GLAD: Multi-language GL/GLES/EGL/GLX/WGL loader-generator [Software]. Retrieved from <https://glad.dav1d.de/>
- Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1990). Computer graphics: Principles and practice (2nd ed.). Addison-Wesley.